Entity Relationship Diagram

Classroom operations, individual learning and family access | Version 1.2

Purpose

This document defines KodeNexus's logical data model and the ownership boundaries needed to connect it to Kodecoon's consolidated data. It is prepared for coordination between the KodeNexus builder and the analytics builder.

The core relationship

A center contains class-code groups and their scheduled time slots. Students join these groups, while each student's module and lesson progress remains individual. Students in the same time slot can study entirely different modules. A schedule change must preserve each learner's history.

One record for shared information

Reuse canonical student, center, class-code and enrollment records from the consolidated registry. KodeNexus owns operational edits; Finance owns money and receipts; Analytics reads authorized views. Each fact has one write owner even when apps use the same database.

How to read the ERD

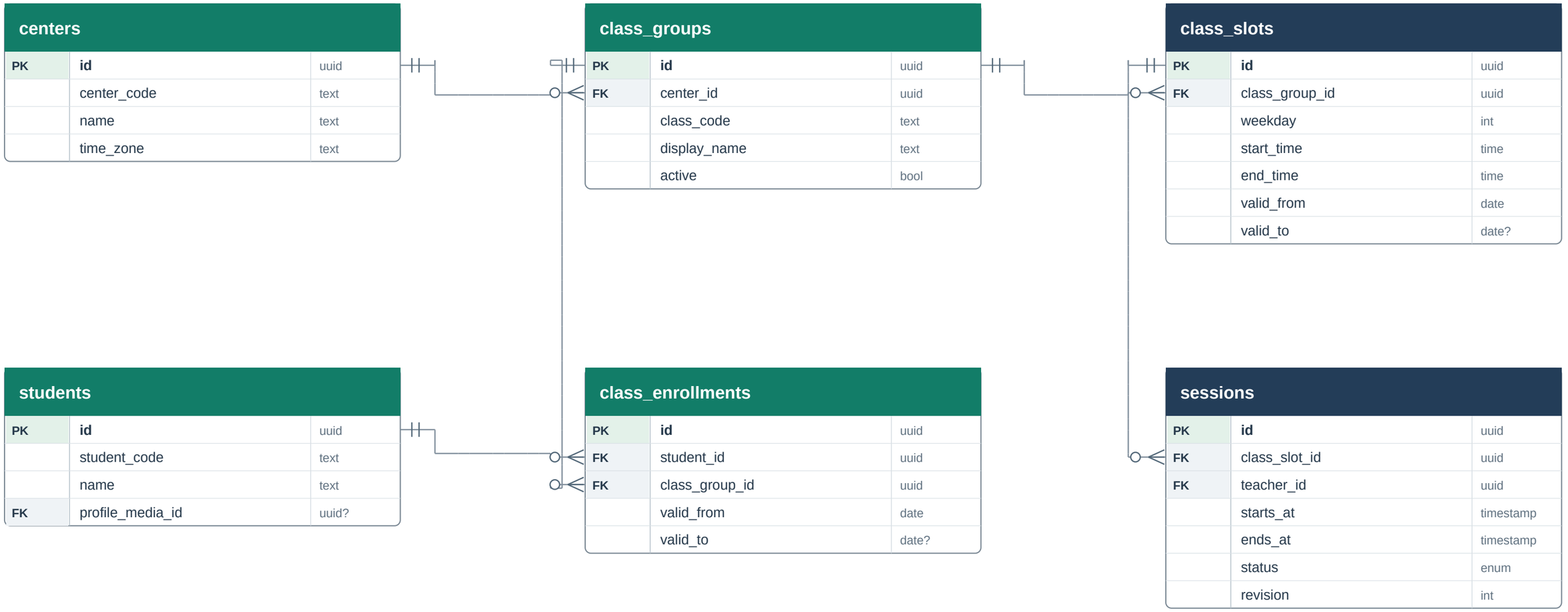
Each table lists its keys and principal fields. PK is its primary key; FK references another table. Crow's-foot connectors show relationship counts. Green tables belong to the shared registry; navy tables belong to KodeNexus. Repeated tables marked [ref] are the same entity shown on another sheet. A question mark after a type means nullable.

Scope of the handoff

This is a logical interface specification. Physical table names and identifiers are mapped to the colleague's consolidated database during integration. It does not certify deployment. The companion relationship register identifies every FK, including links across diagram sheets. Financial amounts, receipts and bank details remain outside KodeNexus.

01 / Centers, class codes and enrollment

Class-code identity and scheduling are separate from individual curriculum.



PK Primary key FK Foreign key || Exactly one o| Zero or one o< Zero or many

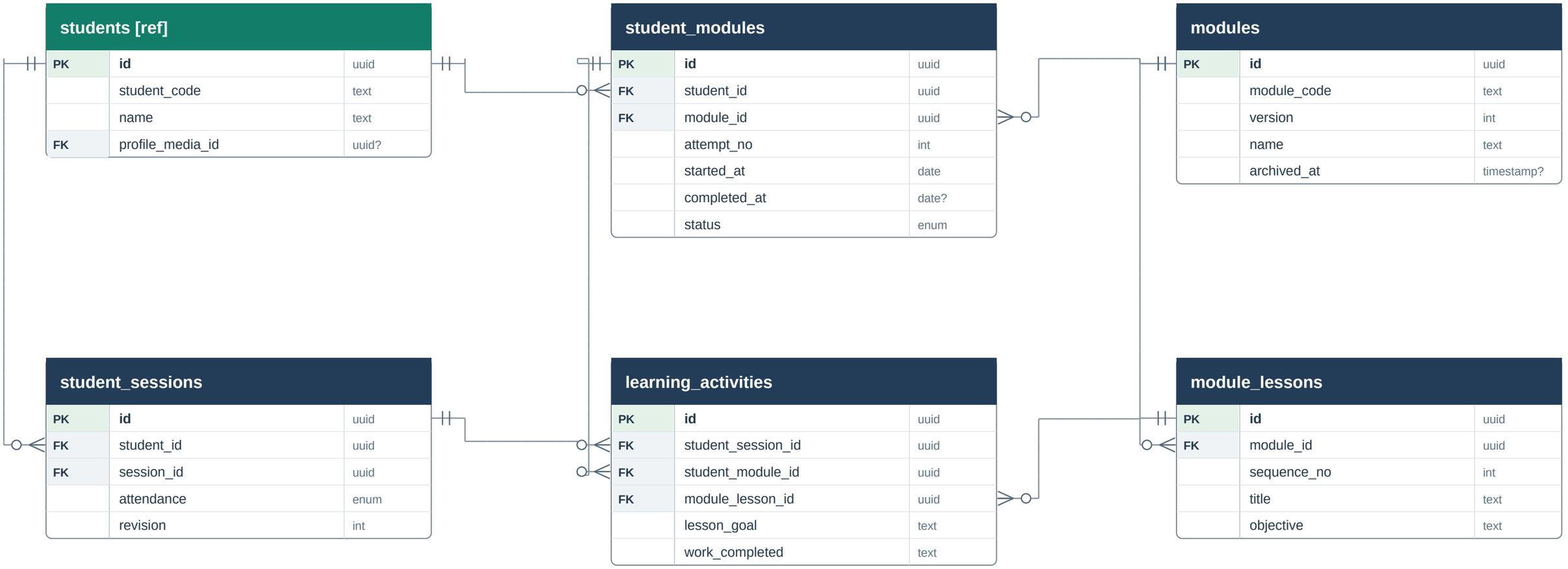
GREEN Shared registry NAVY KodeNexus [ref] Entity repeated from another sheet

Keys: center_code is unique; class_code is unique within its center. An enrollment records a dated period in a class group. Slot validity records schedule changes; session start/end timestamps describe the actual occurrence.

Do not erase later enrollment changes when correcting an earlier date. Make-up attendance is recorded in student_sessions. The center time zone controls timetable display; the database stores actual session timestamps consistently.

02 / Individual modules and class activity

Different learners in one time slot can follow different modules and lessons.



PK Primary key FK Foreign key || Exactly one o| Zero or one o< Zero or many

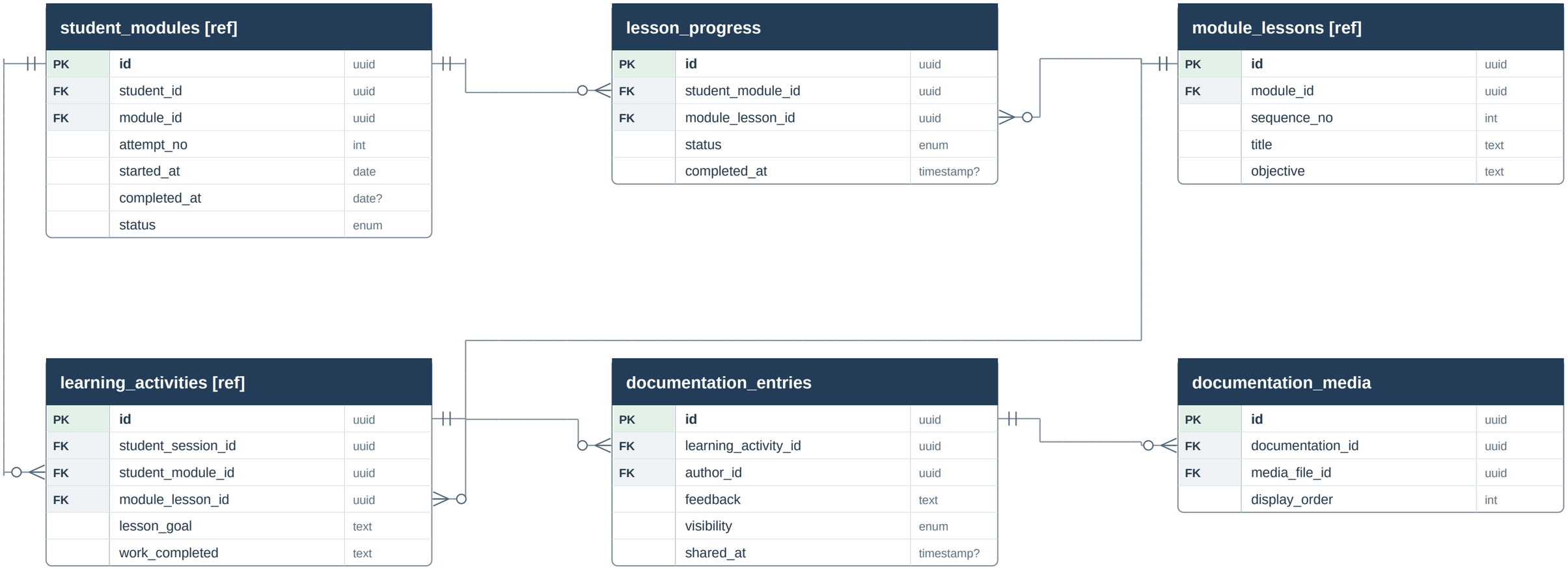
GREEN Shared registry NAVY KodeNexus [ref] Entity repeated from another sheet

Keys: UNIQUE(student_id, module_id, attempt_no), UNIQUE(module_id, sequence_no), UNIQUE(student_id, session_id). A repeated module creates a new student_modules attempt. A session may contain several learning activities.

Validate that the activity's student_module belongs to the attending student, and its lesson belongs to that module. Moving the student to another class group must preserve module attempts, activities and learning history.

03 / Progress and class documentation

Learning records stay attached to the student, module attempt, lesson and class date.



PK Primary key FK Foreign key || Exactly one o| Zero or one o< Zero or many

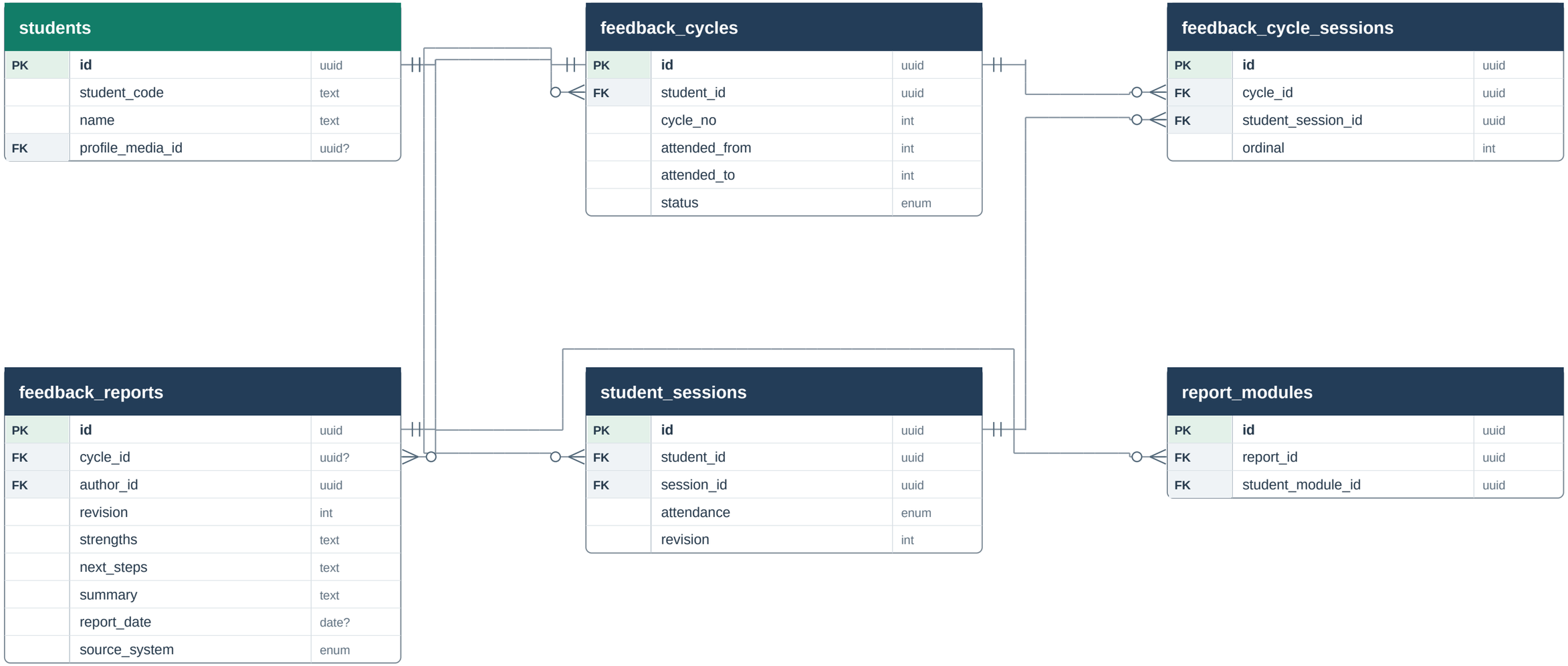
GREEN Shared registry NAVY KodeNexus [ref] Entity repeated from another sheet

Keys: UNIQUE(student_module_id, module_lesson_id) for progress; UNIQUE(documentation_id, media_file_id) for attachments. Documentation contains short class feedback; media links attach Drive photos/videos without duplicating the underlying file.

Only shared documentation is available to linked families. Staff notes remain private. Module-end assessment is stored separately from class documentation. Media access must be checked against the documentation's learner.

04 / Feedback after every 12 attended classes

Report cycles follow the learner across modules and centers; module completion is not required.



PK Primary key FK Foreign key || Exactly one o| Zero or one o< Zero or many

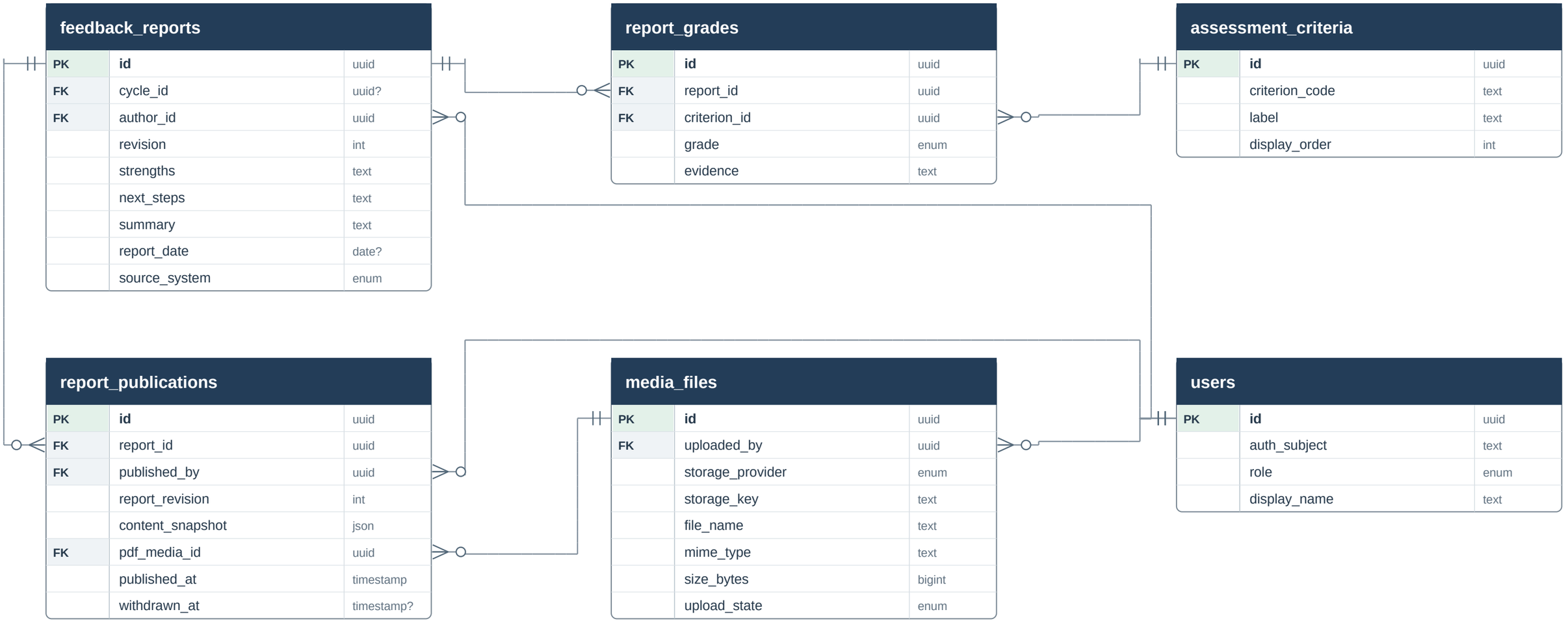
GREEN Shared registry NAVY KodeNexus [ref] Entity repeated from another sheet

UNIQUE(student_id, cycle_no), UNIQUE(student_session_id), UNIQUE(cycle_id, ordinal). Only Present sessions count: cycle 1 covers 1-12, cycle 2 covers 13-24. Leave and Absent do not count. Corrections after publication require an audited revision, never silent rewriting of a published PDF.

A report may cover multiple modules in the same attendance cycle. report_modules links that context; it does not make module completion a condition for feedback. The family sees due / not yet published / available, plus previous report versions. Earlier Tracker feedback is imported as dated historical feedback with no attendance cycle; it must not be counted as a new 12-Present-session report.

05 / Report grades and published PDFs

Staff generate or upload a PDF; parents download the published version and see its history.



PK Primary key FK Foreign key || Exactly one o| Zero or one o< Zero or many

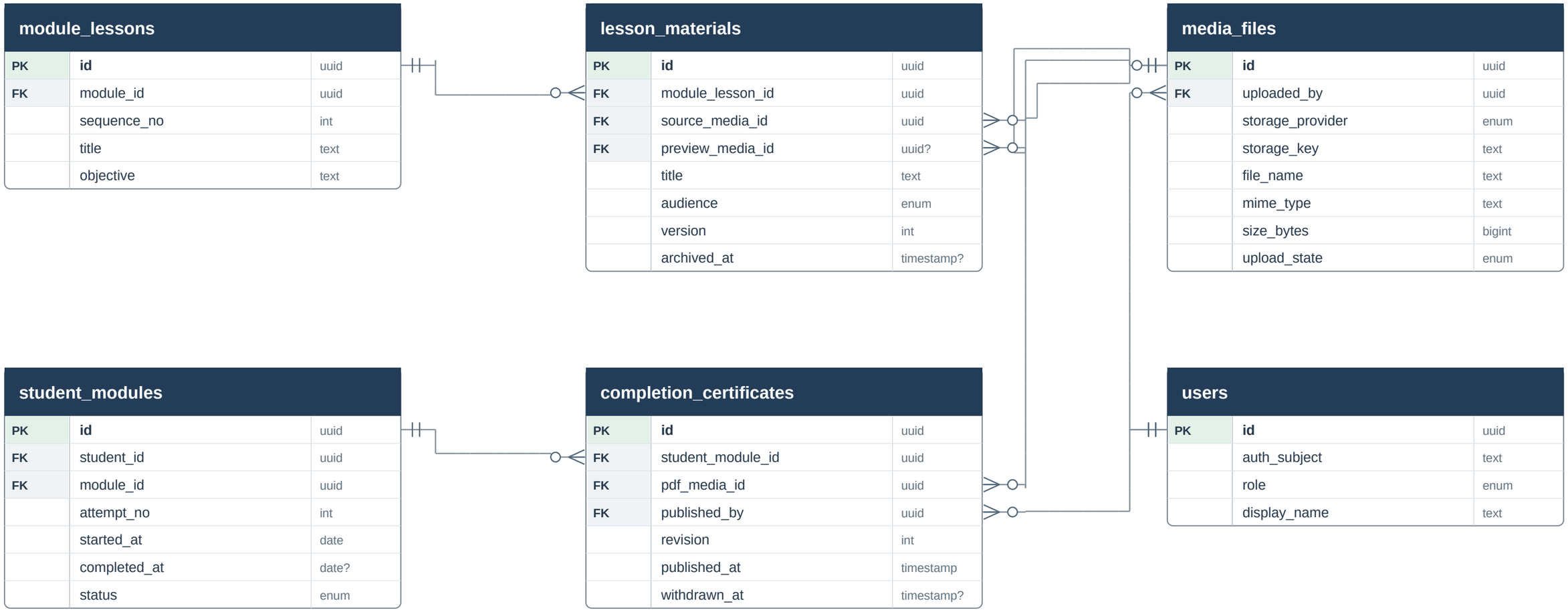
GREEN Shared registry NAVY KodeNexus [ref] Entity repeated from another sheet

UNIQUE(report_id, criterion_id), UNIQUE(report_id, report_revision). Criteria: coding, building/designing, creativity, problem solving, communication. Grades: S, A+, A, A-, B+, B, B-. S means competition-ready mastery; the PDF includes a polite grade legend.

The publication freezes student/module names, grades, feedback and Kodecoon branding. pdf_media_id points to the generated or uploaded private PDF. Drafts are staff-only; linked parents see published, non-withdrawn versions. Uploaded reports need not have structured grades.

06 / Lesson resources and completion certificates

Worksheets, PDF previews and original slide decks are resources; certificates follow completed modules.



PK Primary key FK Foreign key || Exactly one o| Zero or one o< Zero or many

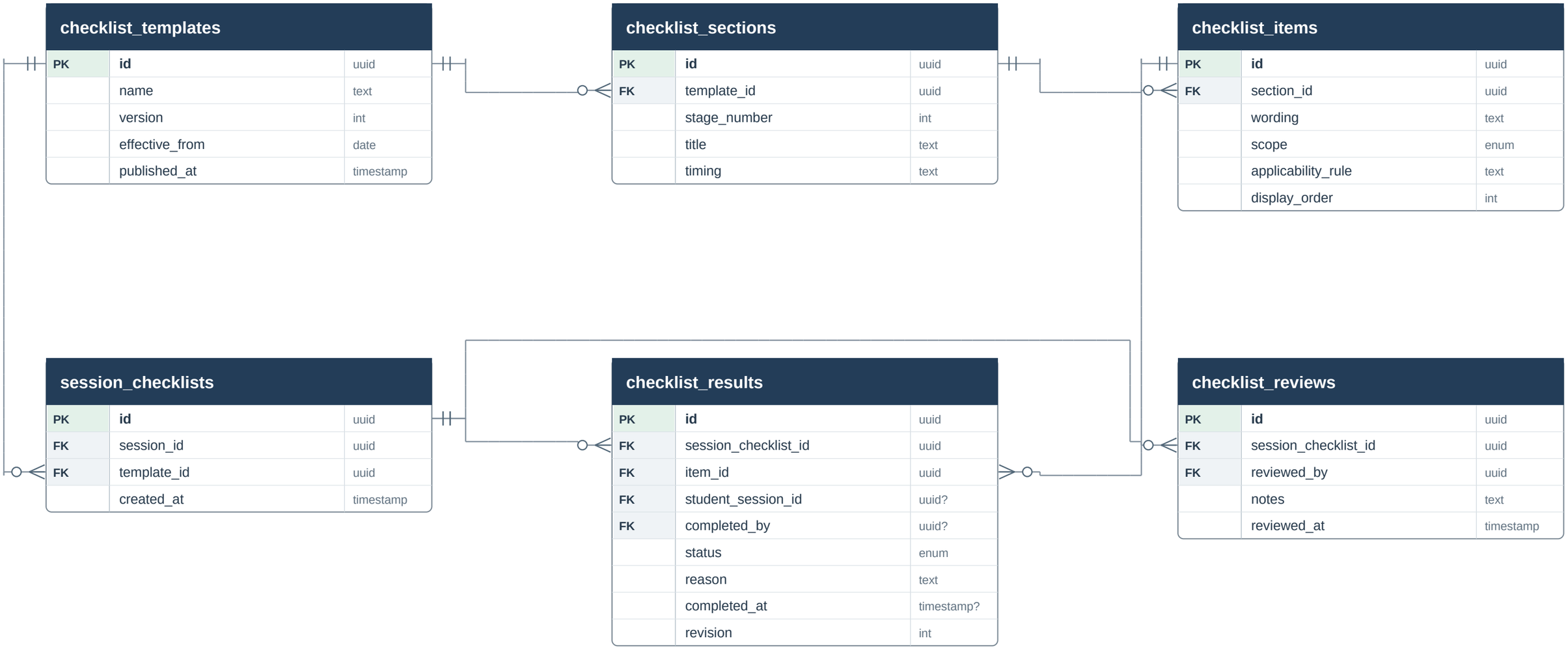
GREEN Shared registry NAVY KodeNexus [ref] Entity repeated from another sheet

Lesson files attach to an individual lesson. source_media_id holds the original PDF/PPT/PPTX; preview_media_id optionally holds a PDF preview. Audience defaults to staff; family access requires an explicit release and a matching learner/module entitlement. Preserve version history.

A certificate requires a completed student_module, independently of the 12-class feedback cycle. Staff upload its PDF; linked parents can download published versions. UNIQUE(student_module_id, revision). storage_provider + storage_key identifies a private object, not a public URL.

07 / Teacher checklist and supervision

Class tasks count once; individual-student follow-ups count per applicable learner.



PK Primary key FK Foreign key || Exactly one o| Zero or one o< Zero or many

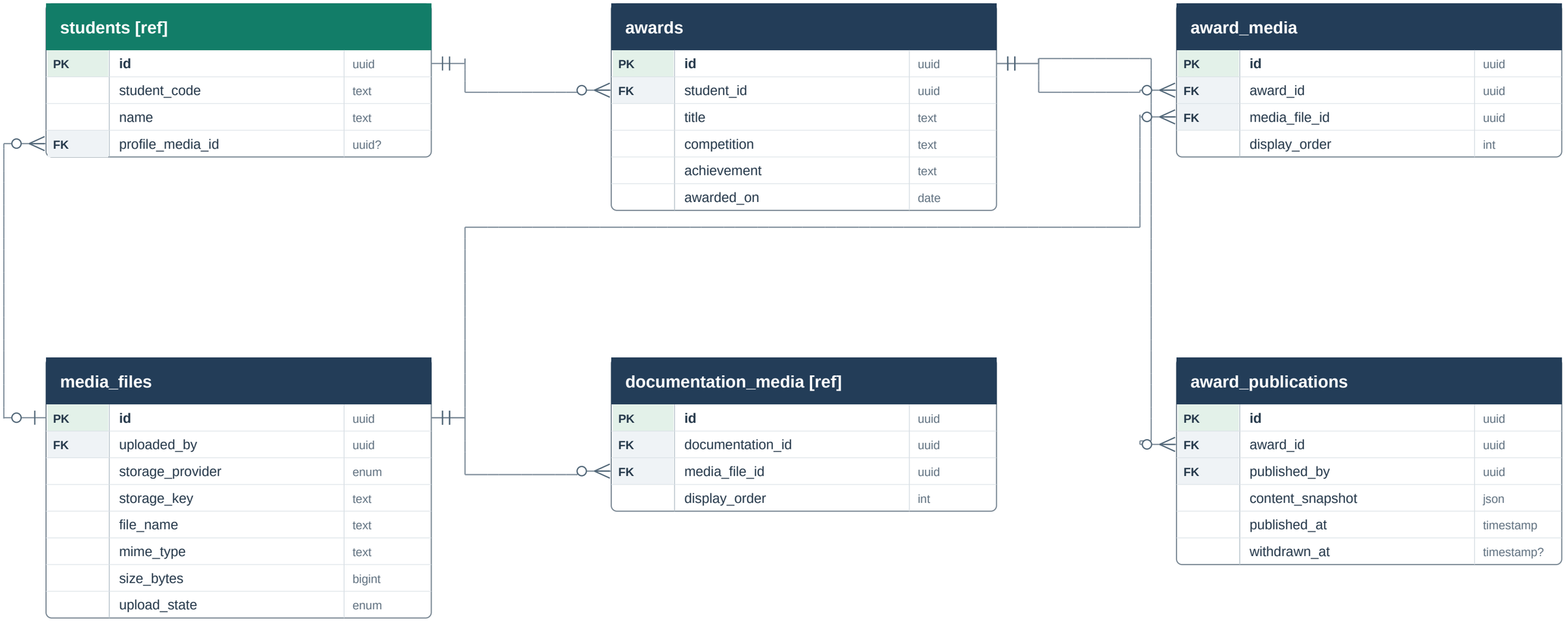
GREEN Shared registry NAVY KodeNexus [ref] Entity repeated from another sheet

Source checklist: 38 checkboxes across eight stages (7/4/5/8/3/5/3/3). Pin a published template version per session. student_session_id is null for class tasks and required for student tasks; enforce one result per applicable item/learner.

Completion = done / applicable required items. N/A requires a reason and is excluded. Pending, overdue and not-yet-due remain distinguishable. Supervisor views compute center -> time slot -> student detail from these records, without a second editable checklist copy.

08 / Photos, videos and Hall of Fame

Academy-controlled Google Drive stores student media; KodeNexus stores access-scoped file references.



PK Primary key FK Foreign key || Exactly one o| Zero or one o< Zero or many

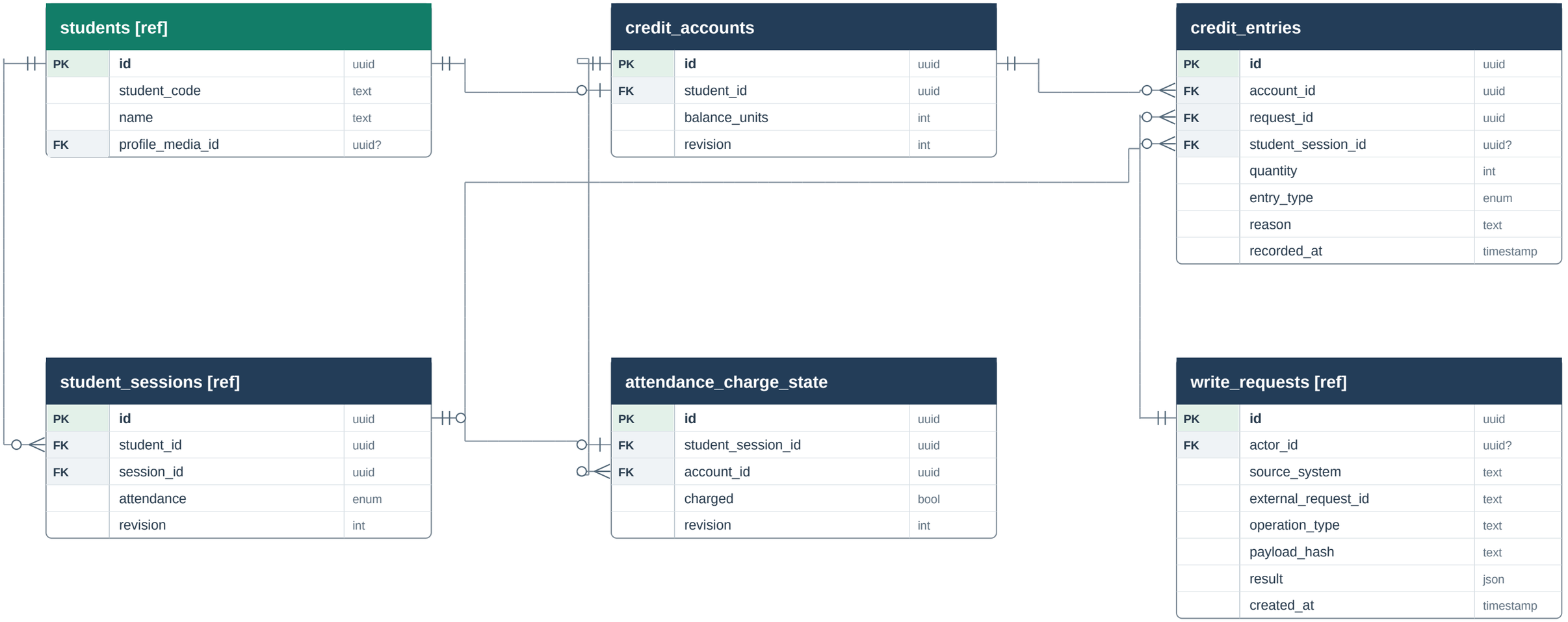
GREEN Shared registry NAVY KodeNexus [ref] Entity repeated from another sheet

Keys: UNIQUE(drive_file_id), UNIQUE(award_id, media_file_id). A single stored file can be linked to documentation or an award. students.profile_media_id identifies the profile portrait. An upload must be finalized before family access.

Private playback must work for a parent logged into KodeNexus without the teacher's Google login. Authorize every file from its linked child and shared record. Award publications expose selected achievement content and associated album images; staff retain editing controls.

09 / Credit units and attendance charges

One authoritative credit ledger; payment amounts and banking information stay external.



PK Primary key FK Foreign key || Exactly one o| Zero or one o< Zero or many

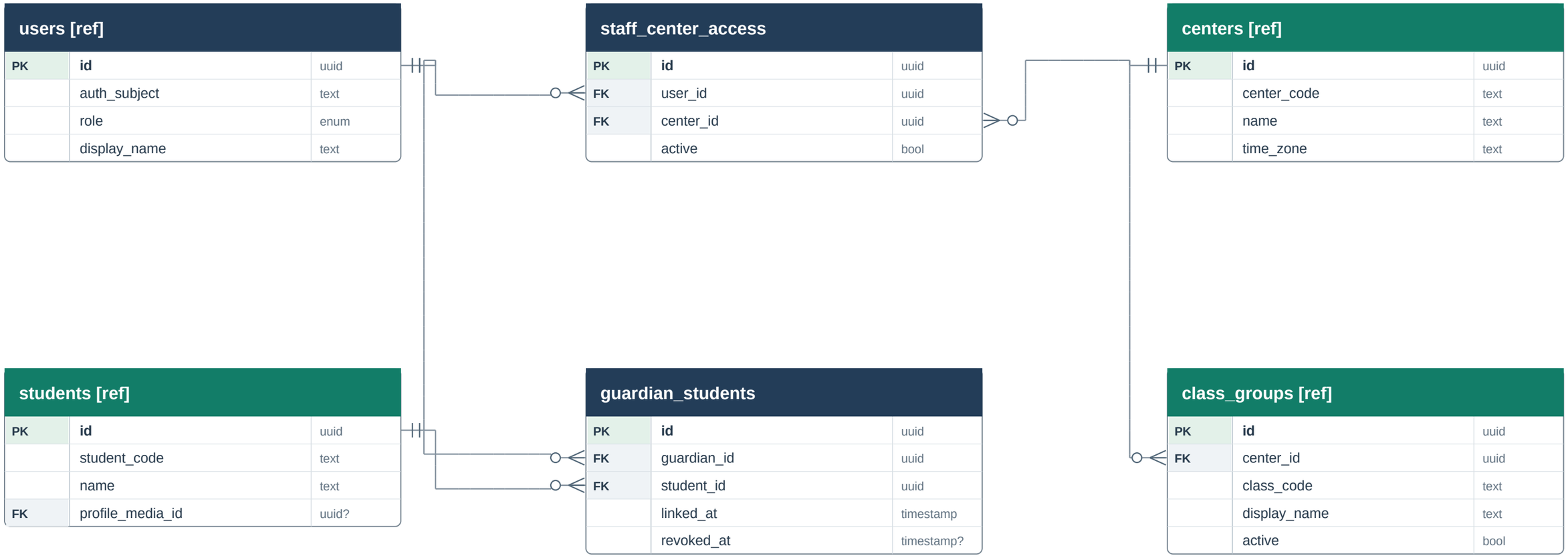
GREEN Shared registry NAVY KodeNexus [ref] Entity repeated from another sheet

Keys: UNIQUE(student_id) for accounts and UNIQUE(student_session_id) for charge state. balance_units must equal the ledger sum and stay non-negative. Ledger entries are append-only; corrections add compensating entries.

An authorized finance grant sends canonical student ID, credit units and a unique operation reference, without payment details. KodeNexus records attendance use/refunds atomically with the balance and request receipt. A retry must never create a second deduction or grant.

11 / Staff and family authorization

Teacher and center manager have equal staff permissions; family access is child-specific.



PK Primary key FK Foreign key || Exactly one o| Zero or one o< Zero or many

GREEN Shared registry NAVY KodeNexus [ref] Entity repeated from another sheet

Keys: UNIQUE(auth_subject), UNIQUE(user_id, center_id) and at most one active guardian/student link. Staff-center membership grants the center access needed for the academy-wide timetable. Both staff roles use the same permission set.

Parents can view their linked child's shared documentation, progress, awards, credit-unit history, published feedback PDFs and certificates. Deny peer access, private notes, staff checklists and finance records in pages, APIs and database rules. No assignment/submission workflow is in scope.

11 / Legacy data-source mapping

The Tracker and OClass are migration lineage; use one reconciled canonical dataset after cutover.

Legacy source and fields	Map to KodeNexus logical records	Cutover handling and ownership
Student Progress Tracker: student list and identifiers present in the sheet	students (shared canonical student ID)	Use the colleague's canonical student IDs. Keep a source-ID crosswalk; do not identify or merge learners by name alone.
Student Progress Tracker: class code, learner roster, center and scheduled time where recorded	class_groups, class_slots, class_enrollments	Reconcile each class code and dated membership once. KodeNexus owns ongoing classroom schedule and roster actions through the agreed shared-data interface.
Student Progress Tracker: current credits left	credit_accounts.balance_units plus a reconciled opening credit_entries record	Confirm the unit and cutover date. Compare against OClass before loading; after cutover the KodeNexus credit ledger is the only classroom credit balance.
Student Progress Tracker: previous end-module feedback/report	feedback_reports (source_system + report_date); link report_modules only where module identity is verified	Preserve original date and text as historical feedback. It may have no 12-attendance cycle and must not count toward a new feedback cycle. Do not invent missing grades or fields.
OClass: overlapping student list, class code, roster and credits	The same students, class_groups, class_slots, class_enrollments and credit_accounts records	OClass is a legacy comparison/import source, not a second live writer. Resolve differences against the colleague's consolidated values and record unresolved conflicts before cutover; never silently overwrite one source with the other.
OClass: payment received, IDR amounts, payment history and receipt/money details	Outside KodeNexus; Finance owns financial records	Do not import money or receipts into KodeNexus. If retained, map them to the separate protected Finance system; the Finance schema is not defined by this KodeNexus ERD.

The source sheet headers and the colleague's migrated schema were not provided for inspection. This is a field-level logical mapping based on Gio's descriptions, not a verified import manifest. Confirm exact columns, identifiers, balances, dates and conflict resolution with the colleague. After reconciliation, analytics and KodeNexus should consume the one canonical shared dataset; neither spreadsheet remains an editable parallel source.

13 / Responsibility for data changes

One authoritative record per topic; separate apps can use the same data safely.

Data	Authoritative home and write responsibility	Analytics / family access
Students, centers and class codes	Shared registry using the colleague's consolidated canonical IDs. KodeNexus submits validated additions/edits through the integration interface. Shared core retains identity ownership.	Analytics reads the shared records. Families receive permitted fields for linked children only.
Enrollment and schedules	Shared enrollment history; KodeNexus manages staff scheduling actions and dated sessions. Reconcile class codes and schedule records once; avoid duplicate editable rosters.	Analytics reads dated records and aggregates. Family profile shows the child's enrollment history.
Modules, learning and feedback	KodeNexus writes modules, individual learner progress, lesson resources, after-class documentation, 12-attendance feedback and completion certificates.	Analytics reads approved learning views. Families receive child-specific published records only.
Files and awards	KodeNexus owns metadata and access rules. Academy-controlled Google Drive holds files under the agreed storage direction; storage adapters can support private object storage without changing ownership.	Analytics receives only necessary metadata. No public child-media folder or unrestricted listing.
Teacher checklist	KodeNexus owns template versions, completion results and supervisor reviews. The dashboard calculates completion from those results.	Staff-only. Analytics can read approved completion aggregates.
Credit units	One ledger managed by the KodeNexus credit writer. Finance sends authorized grant commands; attendance and staff corrections use controlled ledger operations.	Families see credit units and usage. No amount paid, receipts or bank details enter KodeNexus.
Financial history	The separate Finance app writes payments in IDR, receipts, expenses and reimbursements. Its protected schema is outside KodeNexus access.	Analytics reads approved finance views. KodeNexus receives credit-unit grant commands only, without money or receipts.

These boundaries define what KodeNexus needs and writes. Map shared-core entities to the colleague's actual tables rather than creating copies. Each application uses its own least-privilege account; do not share a broad service-role credential.

14 / Integration contract

Shared registry, KodeNexus, Finance and Analytics: one writer per fact.

Canonical identifiers

Exchange the consolidated schema, primary keys, class-code rules and legacy-ID mappings. Reuse canonical student/center/class IDs; never match students by name alone. KodeNexus-specific records reference those IDs. Guardian authorization comes from verified account links, never email similarity. The Student Progress Tracker and OClass are legacy inputs to reconcile once; the colleague's consolidated database is the canonical integration target after cutover.

Write ownership and change delivery

Agree the endpoint or controlled database operation for each shared-core edit. Assign operation IDs, revision checks and an audit actor to changes. Analytics reads approved views or replicas; it does not independently rewrite the classroom records. Derived dashboards never become a second source of attendance or balances.

Credit opening position and cutover

Reconcile opening credit units and active enrollment dates against the consolidated registry. Deduplicate imports and credit grants by source operation ID. After cutover, one system owns each write; other apps read approved views instead of editing copies.

Relationship register

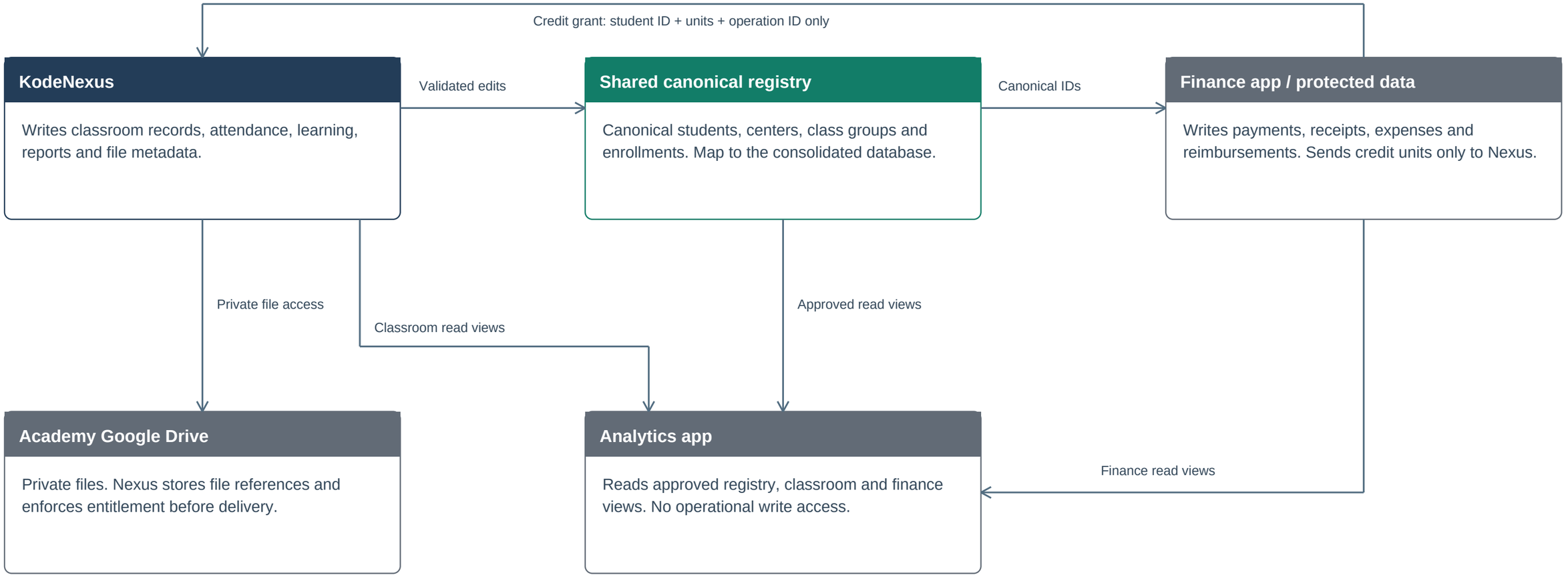
The accompanying relationship-register.csv lists every FK, its target and whether it can be absent. Entity dictionaries list the fields shown in the diagrams. Domain sheets repeat reference tables so each diagram remains readable; repeated names always identify the same table.

Implementation boundary

This document is the KodeNexus logical data contract for integration. It makes no claim that database migrations or live authorization have already been deployed. Physical integration mapping follows comparison with the consolidated schema; no student data or credentials are needed to exchange the ERD.

15 / Application and data-source flow

Apps remain separate. Shared identifiers connect operations, finance and read-only analysis.



Shared source, separate access

The registry box is a logical authority, not a requirement to create another database. Finance sends a separate idempotent grant command to the Nexus credit ledger: canonical student ID, units and operation ID only.

No second editable copy

Analytics may store derived aggregates or a read replica with a freshness timestamp. It does not become the owner of students, attendance, credit balances or payment records.